COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structure and Algorithm	**SECTION:**	CPE 21S2
DATE PERFORMED:	9/18/26	**DATE SUBMITTED:**	9/23/2026
NAME:	Estorco, Clint Hoven A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-On-Activity 9.1 Binary Search Tree

1. PROCEDURE OUTPUT

ILO A	
MAIN CPP	HEADER FILE

main (3).cpp > main()

```cpp
#include <iostream>
#include "BST.h"

int main() {
    // Root node
    Tree<char> tree('A');

    // Insert direct child nodes of A
    Tree<char>* B = tree.addChild('B');
    Tree<char>* C = tree.addChild('C');
    Tree<char>* D = tree.addChild('D');
    Tree<char>* E = tree.addChild('E');
    Tree<char>* F = tree.addChild('F');
    Tree<char>* G = tree.addChild('G');

    // Child of D
    Tree<char>* H = D->addChild('H');

    // Children of E
    Tree<char>* I = E->addChild('I');
    Tree<char>* J = E->addChild('J');

    // Children of F
    Tree<char>* K = F->addChild('K');
    Tree<char>* L = F->addChild('L');
    Tree<char>* M = F->addChild('M');

    // Child of G
    Tree<char>* N = G->addChild('N');

    // Children of J
    Tree<char>* P = J->addChild('P');
    Tree<char>* Q = J->addChild('Q');

    // Traversals
    std::cout << "preorder: ";
    tree.preOrder();
```

```cpp
#ifndef TREE_H
#define TREE_H

template <typename T>
class Tree
{
private:
    // create a key
    T key;
    Tree<T>* firstChild;
    Tree<T>* nextChild;

public:
    // constructor
    Tree(T value);

    // add child
    Tree<T>* addChild(T value);

    // traversing (TRAVERSING TECHNIQUES)
    void preOrder();
    void postOrder();
    void inOrder();

    // searching
    void findData(int CHOICE, T KEY);
};

// Constructor
template <typename T>
Tree<T>::Tree(T value)
{
    key = value;
    firstChild = nullptr;
    nextChild = nullptr;
}
// Add child
```

main (5).cpp > main()

```cpp
    tree.preOrder();
    std::cout << "\ninorder: ";
    tree.inOrder();
    std::cout << "\npostorder: ";
    tree.postOrder();
    std::cout << "\n\n";

    // Table 9-5 Searches
    std::cout << "Find H using preorder: ";
    tree.findData(1, 'H');
    std::cout << '\n';

    std::cout << "Find I using inorder: ";
    tree.findData(2, 'I');
    std::cout << '\n';

    std::cout << "Find Q using postorder: ";
    tree.findData(3, 'Q');
    std::cout << '\n';

    std::cout << "Find Z using preorder: ";
    tree.findData(1, 'Z');
    std::cout << "\n\n";

    // Add O as a child of G
    Tree<char>* O = G->addChild('O');

    std::cout << "Find O using preorder: ";
    tree.findData(1, 'O');
    std::cout << '\n';

    std::cout << "Find O using inorder: ";
    tree.findData(2, 'O');
    std::cout << '\n';

    std::cout << "Find O using postorder: ";
    tree.findData(3, 'O');
    std::cout << '\n';

    return 0;
}
```

```cpp
}
// Add child
template <typename T>
Tree<T>* Tree<T>::addChild(T value)
{
    Tree<T>* newNode = new Tree<T>(value);


    if (firstChild == nullptr){
        firstChild = newNode;
    } else {
        Tree<T>* temp = firstChild;
        while (temp->nextChild != nullptr){
            temp = temp->nextChild;
        }
        temp->nextChild = newNode;
    }

    return newNode;
}

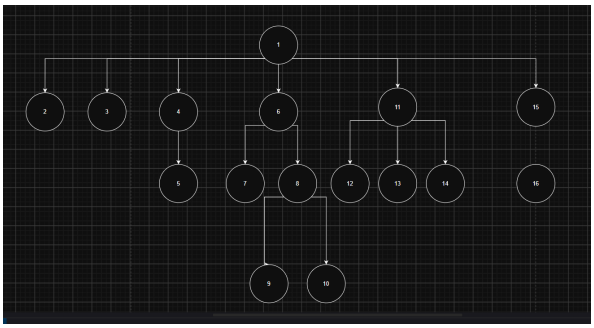

// Preorder
template <typename T>
void Tree<T>::preOrder(){
    // current node
    std::cout << key << " ";
    // first child
    if(firstChild != nullptr){
        firstChild->preOrder();
    }
    if(nextChild != nullptr){
        nextChild->preOrder();
```

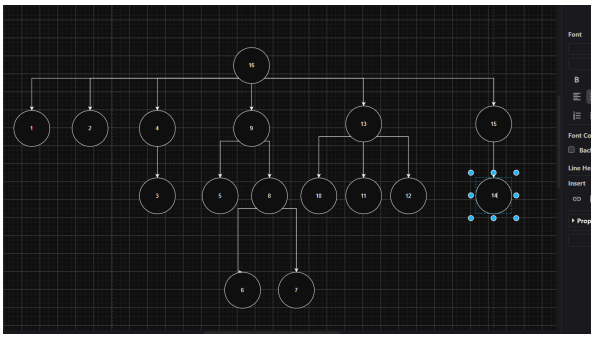
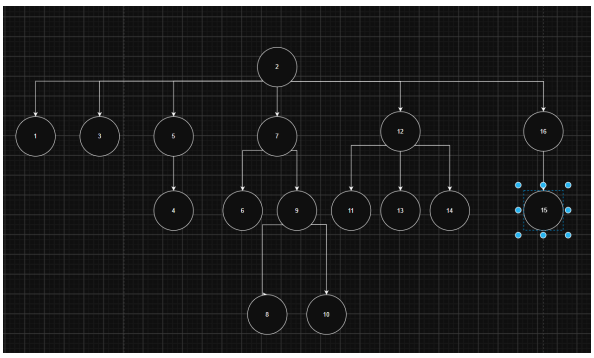
```
68          nextChild->preOrder();
69      }
70  }
71
72
73  // Inorder
74  template <typename T>
75  void Tree<T>::inOrder(){
76      // first child
77      if(firstChild != nullptr){
78          firstChild->inOrder();
79      }
80      // current node
81      std::cout << key << " ";
82      if(nextChild != nullptr){
83          nextChild->inOrder();
84      }
85  }
86
87
88  // Postorder
89  template <typename T>
90  void Tree<T>::postOrder(){
91      Tree<T>* child = firstChild;
92
93      while(child != nullptr){
94          child->postOrder();
95          child = child->nextChild;
96      }
97      // current node
98      std::cout << key << " ";
99  }
100
```

TASK 2 (TABLE)

NODE	HEIGHT	DEPTH
A	3	0
B	0	1
C	0	1
D	1	1

E	2	1
F	1	1
G	10	1
H	0	1
I	1	2
J	0	2
K	0	2
L	0	2
M	0	2
N	0	2
O	0	2
P	0	3
Q	0	3

ILO B	
Pre Order	

Post Order	
In Order	

TASK 3.2

PRE ORDER

```
57
58  // Preorder
59  template <typename T>
60  void Tree<T>::preOrder(){
61      // current node
62      std::cout << key << " ";
63      // first child
64      if(firstChild != nullptr){
65          firstChild->preOrder();
66      }
67      if(nextChild != nullptr){
68          nextChild->preOrder();
69      }
70  }
71
```

The output produced is the sequence of character nodes visited by the character-first traversal scheme used by the print operation, although that sequence can also be called the pre-order traversal sequence of the binary tree that contains all the character nodes.

POST ORDER	

	86
	87
	88 // Postorder
	89 template <typename T>
	90 void Tree<T>::postOrder(){
	91 Tree<T>* child = firstChild;
	92
	93 while(child != nullptr){
	94 child->postOrder();
	95 child = child->nextChild;
	96 }
	97 // current node
	98 std::cout << key << " ";
	99 }

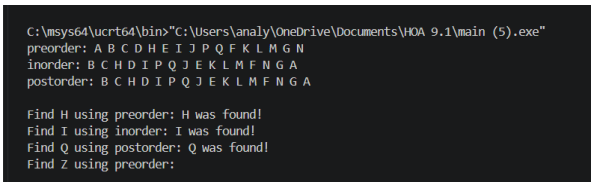
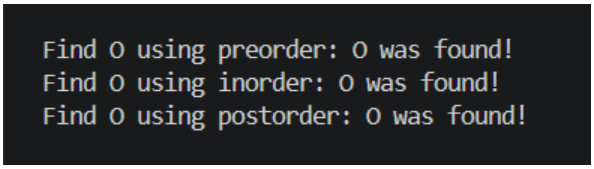
The output function is executed after the conditional guard statements, and after the traversals of the pointers as shown in this sequence; this sequence is the result of a post-order traversal of a binary tree of alphabetic elements from A to Q.	

IN ORDER	

	73 // Inorder
	74 template <typename T>
	75 void Tree<T>::inOrder(){
	76 // first child
	77 if(firstChild != nullptr){
	78 firstChild->inOrder();
	79 }
	80 // current node
	81 std::cout << key << " ";
	82 if(nextChild != nullptr){
	83 nextChild->inOrder();
	84 }
	85 }
	86

As designed, the call to print happens before both the conditional checks and recursive pointer movements in the call stack as you go through this sequence with a binary tree filled with characters A to Q.	

TASK 3.3

OUTPUT:	
EXPLANATION:	findData – Search function with two parameters: Choice, key. Dynamically searches a binary tree for a specific target element (key) using the selected traversal methodology (Choice).
TASK 3.4	
OUTPUT:	
EXPLANATION:	Leaf node 'O' is added under its parent node 'G', and queries are then made using the findData routine: "Execution results as seen, regardless of which kind of the tree traversal technology is used. The node 'O' is identified in each of them, meaning that the choice of traversal technology influences the search order, but not the accessibility of the data or the integrity of the node value"

SUPPLEMENTARY	
INPUT	OUTPUT

```cpp
#include <iostream>
#include "BST.h"

int main() {
    // Root node initialized with the first value: 2
    Tree<int> tree(2);

    // Insert 2, 3, 9, 18, 0, 1, 4, 5 (BST hierarchy)
    Tree<int>* node0  = tree.addChild(0);   // Left child of 2
    Tree<int>* node3  = tree.addChild(3);   // Right child of 2

    Tree<int>* node1  = node0->addChild(1); // Right child of 0

    Tree<int>* node9  = node3->addChild(9); // Right child of 3

    Tree<int>* node4  = node9->addChild(4); // Left child of 9
    Tree<int>* node18 = node9->addChild(18);// Right child of 9

    Tree<int>* node5  = node4->addChild(5); // Right child of 4

    // Traversals
    std::cout << "preorder: ";
    tree.preOrder();
    std::cout << "\ninorder: ";
    tree.inOrder();
    std::cout << "\npostorder: ";
    tree.postOrder();
    std::cout << "\n\n";

    // Searches
    std::cout << "Find 18 using preorder: ";
    tree.findData(1, 18);
    std::cout << '\n';

    std::cout << "Find 0 using inorder: ";
    tree.findData(2, 0);
    std::cout << '\n';

    std::cout << "Find 5 using postorder: ";
    tree.findData(3, 5);
    std::cout << '\n';

    std::cout << "Find 99 using preorder: ";
    tree.findData(1, 99);
    std::cout << "\n\n";

    // Add 15 as a child node
    Tree<int>* node15 = node18->addChild(15);

    std::cout << "Find 15 using preorder: ";
    tree.findData(1, 15);
    std::cout << '\n';

    std::cout << "Find 15 using inorder: ";
    tree.findData(2, 15);
    std::cout << '\n';

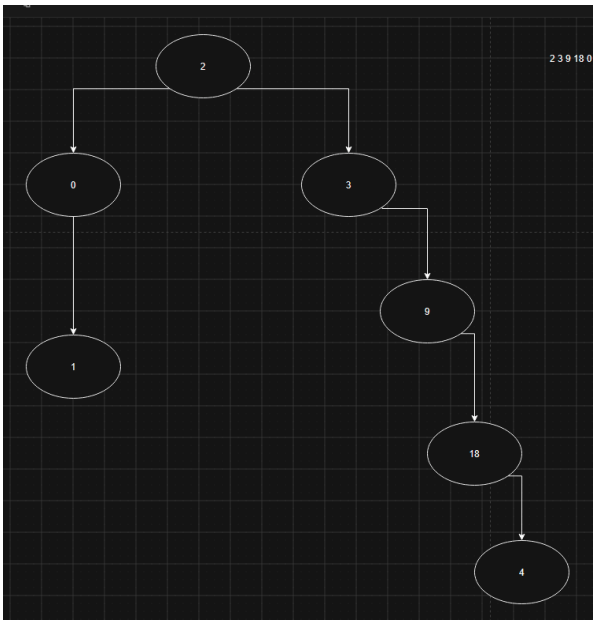
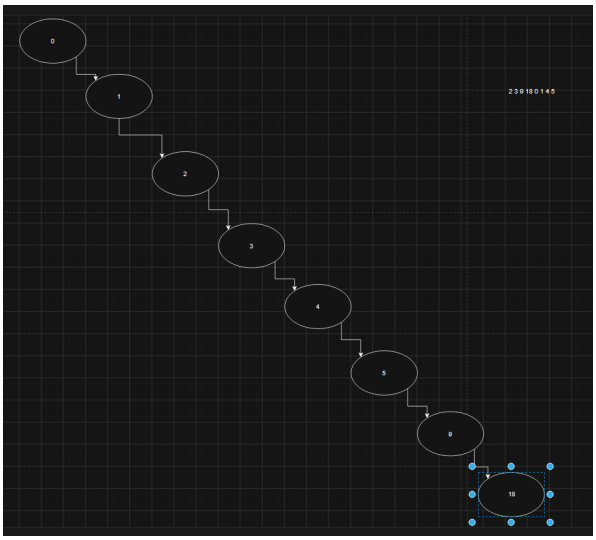
    std::cout << "Find 15 using postorder: ";
    tree.findData(3, 15);
    std::cout << '\n';

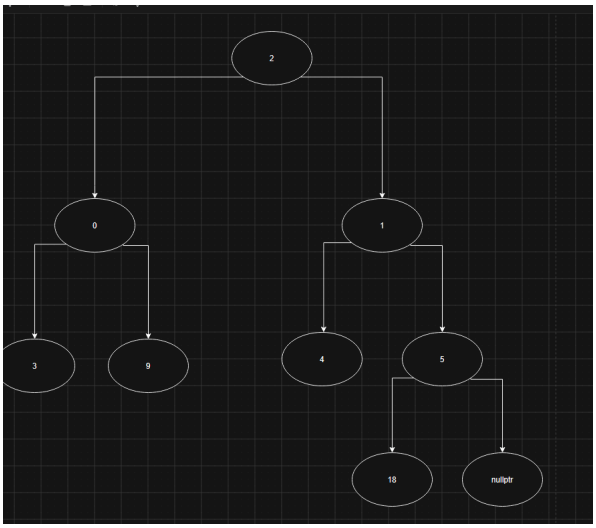
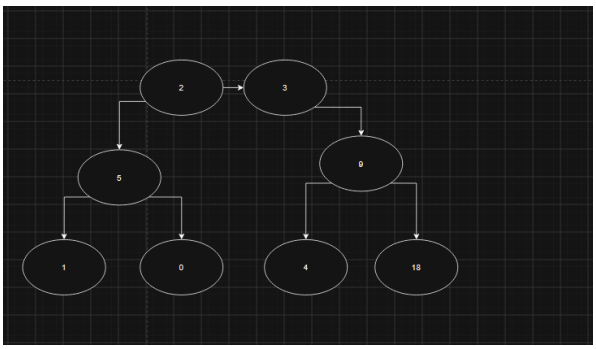
    return 0;
}
```

```
C:\msys64\ucrt64\bin>"C:\Users\ar
preorder: 2 0 1 3 9 4 5 18
inorder: 1 0 5 4 18 9 3 2
postorder: 1 0 5 4 18 9 3 2
```

CONCLUSION:

This module takes a look at the concepts and applications of binary trees in different operations. An understanding of the binary tree mechanics allows to build trees of various structures, to pick various ways for traversing the tree in order to change the way nodes are accessed, and to integrate it into various programs.

Tree Diagram:	
inOrder:	

preOrder:	
postOrder:	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.